

Gradient Methods

Exercise 1

In this exercise, you are provided with the Matlab function **unconstrained_opt.m**, which can be used to solve unconstrained optimization problems. This function allows you to choose between the steepest-descent and the Newton-Raphson methods for optimization.

- (a) Open the function and follow the instructions provided to complete the missing parts of the code so that it functions correctly.
- (b) Use the Matlab function to minimize $f(x) = \alpha x_1^2 + x_2^2$ for $\alpha = 1$ and $\alpha = 100$. Discuss the outcomes of these two cases.
- (c) Verify that applying steepest-descent to find the regression coefficients for the accumulated CO₂ data (from the previous exercises) leads to failure, while the Newton-Raphson method succeeds.
- (d) Explain why the steepest-descent method fails for the CO₂ least-squares problem.

Hint: Consider the condition number of $A^\top A$ in your least-squares problem and recall its definition in terms of the eigenvalues. In Matlab, you can use the command **cond** to compute the conditioning number.

Exercise 2

The Matlab script **SGD.m** implements stochastic gradient descent for a simple linear regression problem. Vary the mini-batch sizes, and then compare the errors obtained when shuffling the data versus not shuffling it.

Exercise 3

A robot estimates its global position $x \in \mathbb{R}^2$ by measuring its distance to three fixed beacons (landmarks) located at known positions $b_1, b_2, b_3 \in \mathbb{R}^2$. These measurements are noisy, and our goal is to find the robot's actual position by solving

$$F(x) = \begin{bmatrix} \|x - b_1\|_2^2 - d_1^2 \\ \|x - b_2\|_2^2 - d_2^2 \\ \|x - b_3\|_2^2 - d_3^2 \end{bmatrix} = 0.$$

This problem can be tackled with the Gauss-Newton algorithm.

■ *Hint:* For $x \in \mathbb{R}^n$, we have $\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$.

- (a) How can this problem be reformulated as an optimization task?
- (b) Follow the instructions in the function **Gauss_Newton.m** to fill in the missing parts of the code.
- (c) In the script **Robot_loc_GN.m**, complete the missing code for the Jacobian $J_f(x)$ of $F(x)$.
- (d) Run the **Robot_loc_GN.m** script and examine the results.